

Lezione 5: Esercitazione Pratica

Collezioni, Interfacce Funzionali, Generics e Stream Avanzate

Michele Gentile

19 Maggio 2026

1 Pipeline di Tracciamento Spedizioni

1. Si modelli la classe **Pacco** definendo i seguenti attributi: **id** (String), **peso** (double), **destinazione** (String) e lo **stato** logistico (gestito tramite un'opportuna enum con valori quali **IN_TRANSITO** e **CONSEGNATO**).
2. Si crei la classe **HubSpedizioni** che contiene e gestisce una collezione di pacchi.
3. Si implementi il metodo `ottieniIdInTransitoOrdinati()` all'interno di *HubSpedizioni* che utilizzi una **pipeline di Stream** per:
 - Selezionare solo ed esclusivamente i pacchi che si trovano nello stato **IN_TRANSITO**.
 - Ordinarli in modalità decrescente in base al rispettivo **peso**.
 - Estrarre unicamente la stringa del loro codice identificativo (**id**).
 - Accumulare il flusso finale all'interno di un **LinkedHashSet** per preservare l'ordine di rimozione dei duplicati e garantire l'unicità.
4. Si estenda l'enum **StatoLogistico** associando a ciascun valore un livello di **priorità** (int). Si implementi poi all'interno di *HubSpedizioni* un metodo che accetti un'interfaccia funzionale come parametro (**Predicate**) per filtrare i pacchi ed effettui un raggruppamento avanzato tramite `Collectors.groupingBy`. Gli elementi all'interno della struttura dati finale dovranno essere ordinati combinando la priorità dello stato logistico e il peso decrescente del pacco attraverso un comparatore composto (`Comparator.thenComparing`).

🔗 Obiettivo: Stream Pipelines, Comparatori e Collezioni Sequenziali

L'obiettivo consiste nel familiarizzare con il concatenamento di operazioni intermedie (filter, sorted, map) e l'utilizzo mirato di collezionisti terminali personalizzati.

⚠️ Attenzione: Preservare l'ordinamento nei Collector

Prestate estrema attenzione alla struttura dati scelta per la terminazione del flusso! Un generico `Collectors.toSet()` non garantisce alcun ordine di iterazione stabile. Se desiderate mantenere l'ordine stabilito tramite l'operazione intermedia `sorted()`, occorre raccogliere i dati esplicitamente in una collezione idonea come **LinkedHashSet**.

2 Hub Logistico Multi-Corriere

1. Si definisca la classe **Spedizione** provvista di una lista interna contenente le descrizioni dei colli (es. `List articoli`).
2. Si implementi la classe **Corriere** che gestisce l'insieme delle spedizioni ad esso assegnate (`List`).
3. Si realizzi la classe di coordinamento generale **PiattaformaLogistica** contenente l'elenco di tutti i corrieri attivi nel circuito.
4. Si scriva il metodo `estraiTuttiArticoliUnici()` che, partendo dall'insieme dei corrieri, utilizzi l'operatore **flatMap** all'interno di una pipeline per appiattare la struttura dati gerarchica (`Corriere → Spedizione → Articolo`), rimuovendo gli elementi duplicati e restituendo un `Set` globale.
5. Si introduca l'enum `ZonaCopertura` (`NAZIONALE`, `INTERNAZIONALE`, `ZTL`) come attributo della classe **Corriere**. Si modifichi successivamente la firma e il comportamento del metodo `estraiTuttiArticoliUnici()` affinché accetti un predicato basato su `ZonaCopertura`, consentendo di estrarre e linearizzare gli articoli solo dai corrieri che operano nelle zone specificate.

🔗 Obiettivo: Strutture Dati Annidate e FlatMap

L'obiettivo è comprendere a fondo lo scenario d'uso d'elezione del metodo `flatMap`: la trasformazione e la linearizzazione di uno `Stream` di collezioni o di gerarchie complesse in uno `Stream` a livello singolo.

💡 Tip: Quando applicare flatMap

Se la funzione di mappatura che intendete passare allo stream restituisce a sua volta uno `Stream` o una collezione secondaria anziché un singolo elemento atomico, `flatMap` rappresenta la scelta corretta per evitare di trovarsi a gestire uno scomodo `Stream<Stream>`.